

Aprendizado por Reforço para Poker *Heads-Up*: os Agentes v15 e v16

1st Ivan Goya Yamasaki
Instituto Tecnológico de Aeronáutica (ITA)
São José dos Campos, Brazil
ivan.yamasaki.9074@ga.ita.br

2nd Amadeu Albuquerque Fontenele Neto
Instituto Tecnológico de Aeronáutica (ITA)
São José dos Campos, Brazil
amadeu.neto.9160@ga.ita.br

Resumo

Este artigo descreve a criação de dois agentes autônomos para um torneio de poker heads-up, v15 e v16, treinados por Aprendizado por Reforço (Reinforcement Learning, RL) com o algoritmo Proximal Policy Optimization (PPO). Diferente das gerações anteriores da linhagem (v1–v14), construídas por heurísticas e ajuste manual, eles aprendem a política diretamente da interação com a engine real. Os dois agentes são idênticos — mesma representação de estado (23 atributos), mesmo espaço de ações (6 ações) e mesmo pool de oponentes — diferindo apenas na recompensa: o v15 usa recompensa densa (fluxo de fichas) e o v16, esparsa (vitória/derrota). Em avaliação greedy de 500 partidas por oponente, ambos superam o melhor heurístico (v14), quebrando a barreira de 70% que o ajuste manual nunca cruzou; o v16 (esparso) é o mais forte, vencendo 399/500 contra o v14 e 1593/2000 (79,7%) contra o conjunto. Discutimos por que a recompensa esparsa, apesar do sinal de aprendizado mais fraco, supera a densa.

Palavras-chave: aprendizado por reforço; PPO; poker heads-up; modelagem de recompensa; agentes autônomos.

Index Terms — reinforcement learning; PPO; heads-up poker; reward shaping; autonomous agents.

1. Introdução

O poker é um problema clássico de tomada de decisão sob incerteza: informação incompleta (as cartas do oponente são ocultas), estocasticidade (distribuição das cartas) e um adversário adaptativo. Essas características o tornam um banco de testes natural para técnicas de

aprendizado de máquina e, em particular, de Aprendizado por Reforço (RL), cuja eficácia em jogos foi demonstrada do Atari [10] a Go e xadrez por *self-play* [11].

Este trabalho documenta o desenvolvimento de dois agentes, v15 e v16, para um torneio de poker *heads-up* (dois jogadores) com estrutura de *blinds* crescentes. Ambos representam uma mudança de paradigma em relação às quatorze gerações anteriores da linhagem (Seção 3): em vez de regras escritas à mão e ajuste manual, a política de decisão é *aprendida* a partir de milhões de mãos simuladas.

As contribuições deste artigo são:

1. Uma visão de alto nível do campeonato e da linhagem de agentes que antecedeu os modelos de RL (Seções 2 and 3);
2. A modelagem do problema como Processo de Decisão de Markov, reusando a engenharia de atributos do melhor agente heurístico (v14) (Seção 4);
3. Um estudo controlado do efeito da função de recompensa: v15 (densa) versus v16 (esparsa), com todo o resto idêntico (Seções 4.5 and 6).

2. O Campeonato

O contexto deste trabalho é um *hackathon* de inteligência artificial para poker. Cada participante submete um *agente autônomo* — um programa que, dado o estado da mesa, decide a jogada — e os agentes se enfrentam em partidas de poker *heads-up* (um contra um). O objetivo é simples de enunciar: ao longo de muitas partidas, **vencer mais do que o adversário**.

Em alto nível, as regras relevantes são: cada partida é *heads-up* (dois agentes, fichas iniciais iguais, vence quem zera o *stack* do oponente); os *blinds* crescem ao longo do jogo, encurtando os *stacks* e forçando decisões cada vez mais arriscadas; cada decisão tem tempo limitado, o que privilegia estratégias eficientes; e, por ser

Trabalho desenvolvido para a disciplina CM-204, Instituto Tecnológico de Aeronáutica (ITA). Código em <https://github.com/IvanYamasaki/HackathonYamadaITAjr>.

estocástico, o desempenho é medido pela *taxa de vitória* em muitas partidas, não num único confronto. O foco adiante é *como* um agente aprende, por conta própria, uma boa política nesse ambiente.

3. A Linhagem Heurística (v1–v14)

Os agentes v15 e v16 não nasceram do zero conceitual: herdam a *representação de conhecimento* acumulada ao longo de quatorze gerações de agentes heurísticos. Cada versão não foi um mero ajuste para “vencer a anterior”: foi ou uma *tentativa de uma estratégia diferente*, ou a *correção de um problema concreto detectado na versão anterior* (identificado por análise de fluxo de fichas e varreduras de hiperparâmetros, os *sweeps*). Esta seção conta a breve história de quatro marcos dessa linhagem — **v1**, **v8**, **v13** e **v14** — com sua lógica de criação e o desempenho nas duas fases oficiais do *hackathon*. O torneio teve duas fases em formato *round-robin heads-up* (todos contra todos, 2000 partidas por confronto): a **Fase 1** classificou a metade superior; na **Fase 2**, os códigos de todos foram revelados e os classificados puderam atualizar seus *bots*. Os agentes desta linhagem compartilhada competiram nas duas fases: o v8 e o v1 na **Fase 1**, o v14 e o v13 na **Fase 2**. Notavelmente, v14 e v13 se enfrentaram na decisão da Fase 2 — o v14 venceu o v13 no confronto direto.

3.1. v1 — o jogador profissional destilado e o *benchmark*

O v1 foi a primeira geração: um bot heurístico que tenta *imitar a lógica de um jogador humano profissional*. Em vez de buscar a solução pela teoria dos jogos, ele codifica diretamente o “livro” do jogador experiente — um conjunto de resultados conhecidos sobre quais situações são boas ou ruins. A mão é classificada por força aproximada (*equity heads-up* contra mão aleatória) e a decisão segue regras de bom senso por categoria:

- **Pré-flop:** abre $3 \times BB$ com mãos *premium*, $2,5 \times$ com boas mãos, *limp* com marginais e desiste com lixo (somente como *small blind*); *3-bet* com *premium*, *call* com mãos fortes e *fold* com lixo ao enfrentar aumento.
- **Pós-flop:** avalia a força real (*rank + kicker*) e separa as mãos em *monster* (aposta de valor a 75% do *pot*), *strong* (60%), *medium* (controle de *pot*) e *weak* (*check-fold*, com *semi-bluff* em *draws* fortes).
- **Stacks curtos** ($\leq 15 BB$): regime *push/fold*; defesa contra *all-in* por força mínima dependente das *pot odds*; aleatoriedade controlada (*slow-play* 15%, *3-bet light* 20%, *bluff* 10%).

Apesar de simples, o v1 é um adversário sólido. Em um teste informal de cinco partidas contra um jogador humano não profissional, o v1 venceu cerca de **80%** das partidas. Submetido à **Fase 1** do *hackathon* (17 *bots*), terminou em **3º lugar**, com **63,7%** de taxa de vitória geral, vencendo 13 dos 16 confrontos. Por combinar comportamento “humano plausível” com desempenho robusto e estável, o v1 foi adotado como o *benchmark* de referência de toda a linhagem — inclusive como um dos oponentes fixos no treino dos agentes de RL (Seção 5.1).

3.2. v8 — destilação prática do estado da arte

O v8 é um salto qualitativo: em vez de regras estáticas, busca *destilar o pacote conceitual* dos agentes de ponta para *No-Limit Hold'em heads-up* (estilo *Libratus* [5] / *Pluribus* [4]) dentro do orçamento severo de tempo do torneio (≈ 50 ms por decisão). Como rodar *Counterfactual Regret Minimization* (CFR) [6] completo é inviável nesse orçamento, o v8 aproxima cada componente por uma alternativa prática:

1. **Equity por Monte Carlo com orçamento de tempo:** a cada decisão pós-flop, amostra mãos do oponente compatíveis com a história de ações e estima a *equity* por simulação, abortando ao se aproximar do *deadline* (≈ 22 ms para o MC, ≈ 38 ms de teto global).
2. **Modelagem de oponente persistente:** rastreia *VPIP*, *PFR* e *aggression factor* (AF) ao longo de toda a partida e adapta os *ranges* e decisões *online*.
3. **Estreitamento de range:** usa as ações desta mão e o perfil persistente para reduzir o espaço de mãos plausíveis do oponente na simulação.
4. **Abstração de ações com mistura de EV:** avalia ações candidatas $\{fold, call, \frac{2}{3}\text{-pot}, pot, overbet\}$ e escolhe a de maior valor esperado, com uma estratégia mista ($\approx 10\%$) para resistir à exploração.
5. **Push/fold de Nash** para *stacks* curtos ($\leq 12 BB$), aproximando a tabela de Nash *heads-up* — essencial porque os *blinds* dobram periodicamente.
6. **Bluff-catching vs. aposta de valor** e uma **rede de segurança**: se o MC não amostrar o suficiente, cai para uma heurística rápida, garantindo zero *time-outs*.

Em resumo, o v8 troca o “livro” fixo do v1 por uma estimativa de *equity* calculada em tempo real e adaptada ao oponente. Foi a versão submetida à **Fase 1**, onde **sagrou-se campeão: 1º lugar** entre os 17 *bots*, com

68,4% de taxa de vitória geral e 15 dos 16 confrontos vencidos — perdeu por margem mínima apenas um confronto. A Tabela 1 resume.

Tabela 1. Fase 1 (round-robin com 17 bots, 2000 partidas/confronto) — destaque para os agentes da linhagem.

Posição	Agente	Confrontos vencidos	WR geral
1º	v8	15/16	68,4%
3º	v1 (benchmark)	13/16	63,7%

(demais participantes omitidos)

3.3. v13 — exploração dirigida do oponente

O **v13** marca a virada para a **Fase 2**, quando os códigos adversários foram revelados e tornou-se possível *explorar fraquezas específicas*. Em vez de buscar robustez genérica, o v13 foi construído por *contra-exploração* dos oponentes mais fortes (incluindo gerações anteriores da própria linhagem). Sua lógica de criação combina:

- **Avaliação fina da mão:** estima a probabilidade de estar à frente de uma mão aleatória por avaliação exata, com classificação detalhada (*overpair*, *top pair* + *kicker*, par do *board*, *flush* do *board*, ...), bônus por *draw* e descontos por textura perigosa do *board*.
- **Defesa mais seletiva contra agressão real:** como os aumentos dos oponentes-alvo eram quase sempre de valor, o v13 desconta a força da própria mão diante de agressão e abandona mãos médias com mais disciplina.
- **Dimensionamento de valor calibrado:** aumentos de valor menores, ajustados para ficar logo abaixo dos limiares de pagamento dos oponentes, extraíndo mais fichas ao longo da mão.
- **Trackers de oponente:** estimativas de *fold-to-raise* e de frequência de aumento que adaptam blefes e *call-downs*; regime *push/fold* domina o *endgame* de *blinds* altos. Decisão em menos de 1 ms.

Submetido à **Fase 2** (8 finalistas), o v13 terminou em **2º lugar (vice-campeão)**, com **64,2%** de taxa de vitória geral e 6 dos 7 confrontos vencidos — perdendo apenas para o sucessor direto, o v14.

3.4. v14 — o campeão e o teto da abordagem heurística

O **v14** é o ponto culminante da linhagem heurística e o **campeão da Fase 2: 1º lugar** entre os 8 finalistas,

com **71,4%** de taxa de vitória geral e **vitória em todos os 7 confrontos** — inclusive batendo o v13 em 60,5% no confronto direto. A Tabela 2 resume o pódio.

Em termos de lógica de criação, o v14 levou a contra-exploração do v13 a um novo patamar, com duas inovações centrais:

- **Agressão controlada (*blitz*):** aumentos frequentes do tamanho do *pot* sob um *orçamento auto-regulado*, que pressiona o oponente sem se expor a contragolpes caros.
- **Endgame ciente de *range*:** ao fim da partida (*stacks* curtos), replica os limiares de *all-in* do oponente e decide *call/shove* por *equity* estimada, com margens conservadoras.

Tabela 2. Fase 2 (round-robin com 8 finalistas, 2000 partidas/confronto) — topo do *leaderboard*.

Posição	Agente	Confrontos vencidos	WR geral
1º	v14	7/7	71,4%
2º	v13	6/7	64,2%

Apesar do título, o v14 evidenciou um *teto estrutural*: como o resultado da partida é decidido em grande parte nas últimas dezenas de mãos (loteria de *all-ins* no regime *push/fold*), o ganho marginal de mais ajuste manual tornou-se decrescente. **É exatamente esse teto que motiva o restante deste trabalho: a partir daqui, abandonamos o ajuste manual e usamos Aprendizado por Reforço para treinar uma nova geração — os agentes v15 e v16 — com o objetivo explícito de vencer o v14.**

O conhecimento acumulado não é descartado: ele está codificado nos 23 atributos do estado (Seção 4.3), em especial no avaliador de força de mão herdado do v14. Treinar “a partir do v14” significa treinar *contra* o v14 (e contra v13, v8 e v1), e não carregar pesos — o v14 é heurístico e não possui parâmetros aprendidos.

Em suma, a linhagem percorre três paradigmas cada vez menos manuais: regras de poker codificadas por humanos (v1–v8), regras refinadas por contra-exploração e *sweeps* (v9–v14) e, a partir daqui, uma política *aprendida* por RL (v15–v16) — que reaproveita todo o conhecimento já embutido nas *features* do v14 e deixa o aprendizado apenas com a parte difícil: *o que fazer com aquela informação em cada spot*. O restante deste artigo detalha essa etapa de RL.

4. Metodologia

4.1. Formulação como Processo de Decisão de Markov

Modelamos a decisão do agente como um Processo de Decisão de Markov (MDP) $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, onde $\mathcal{S}$ é o espaço de estados (Seção 4.3), $\mathcal{A}$ é o espaço de ações discreto (Seção 4.4), P é a dinâmica induzida pela *engine* e pelo oponente, R é a recompensa (Seção 4.5) e γ é o fator de desconto. O objetivo é encontrar uma política $\pi_\theta(a | s)$ que maximize o retorno esperado

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]. \quad (1)$$

A Figura 1 detalha o laço fechado desse MDP — da observação do estado s_t à ação a_t , e desta de volta a s_{t+1} via a *engine* — explicitando *onde* cada componente do agente e do ambiente atua e em que ponto entra a *única* diferença entre v15 e v16 (a recompensa r_t).

4.2. Algoritmo de Treino: PPO com GAE

A política é otimizada por *Proximal Policy Optimization* (PPO) [1], um método de gradiente de política que troca a atualização instável do gradiente *vanilla* por um objetivo *clipado*, o qual limita o tamanho do passo a cada *update*:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right], \quad (2)$$

onde $\rho_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ é a razão entre a política nova e a que coletou os dados, $\hat{A}_t$ é a vantagem estimada e ε (o *clip* PPO, 0,2 na Tabela 5) define a região de confiança. O *clipping* impede que uma única atualização afaste demais a política, o que é essencial num ambiente tão ruidoso quanto o poker.

A vantagem é estimada por *Generalized Advantage Estimation* (GAE) [2], que pondera viés contra variância pelo parâmetro λ :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad (3)$$

em que δ_t é o erro temporal (TD) do crítico V_ϕ , γ é o desconto e λ controla o horizonte efetivo do *bootstrap* ($\gamma = 0,999$ e $\lambda = 0,95$ na Tabela 5). O desconto alto ($\gamma \rightarrow 1$) é deliberado: como o desfecho da partida se concentra nas últimas mãos (regime *push/fold*), o crédito precisa propagar-se por horizontes longos.

A perda total minimizada a cada *update* soma três termos — política, valor e um bônus de entropia que

incentiva a exploração —, ponderados pelos coeficientes da Tabela 5:

$$L(\theta, \phi) = \underbrace{-L^{\text{CLIP}}(\theta)}_{\text{política}} + c_v \underbrace{(V_\phi(s_t) - \hat{R}_t)^2}_{\text{valor}} - c_e \underbrace{\mathcal{H}[\pi_\theta(\cdot | s_t)]}_{\text{entropia}}, \quad (4)$$

com alvo de valor $\hat{R}_t$, coeficiente de valor $c_v = 0,5$ e de entropia $c_e = 0,01$. A Eq. (4) “fecha o circuito” com a Tabela 5: cada coeficiente ali listado é o peso de um destes termos. O termo de entropia é especialmente relevante para o v16, cuja recompensa esparsa exige exploração sustentada para descobrir trajetórias vencedoras. Para os fundamentos de RL, ver Sutton e Barto [3].

4.3. Representação do Estado (23 atributos)

O estado é um vetor $s \in \mathbb{R}^{23}$ construído de forma idêntica no treino e na execução do torneio (uma única implementação, garantindo que a política veja exatamente os mesmos atributos em ambos os contextos). Os atributos agrupam-se em quatro blocos:

Tabela 3. Os 23 atributos do vetor de estado.

Índice	Bloco	Descrição
0–3	Força da mão	força total e “feita”, <i>flag</i> de <i>draw</i> forte e de qualquer <i>draw</i>
4–9	Contexto do <i>spot</i>	<i>pot odds</i> , razões de pagamento, <i>stack</i> efetivo (bb), tamanho do <i>pot</i> , <i>stack-to-pot ratio</i> (SPR)
10–16	Posição e rua	indicadores de rua (R1/turn/river), agressão, <i>small blind</i> , fração de <i>stack</i> , primeira ação da mão
17–22	Modelo do oponente	taxa de agressão, <i>fold-to-raise</i> , taxa de <i>big raises</i> , taxa de <i>folds</i> cedo, aposta corrente (bb), mãos jogadas

O bloco de força de mão é calculado por um avaliador exaustivo de cinco cartas, idêntico ao do v14, que categoriza a mão (par, dois pares, trinca, ...) e atribui uma força contínua em $[0, 1]$, com bônus por *draws* (projetos de *flush*/sequência).

4.4. Espaço de Ações

O agente escolhe entre seis ações discretas, posteriormente traduzidas para o inteiro que a *engine* espera (−1 desistir, 0 pagar/*check*, $N > 0$ subir para o total N):

Agente π_θ (decide a jogada)

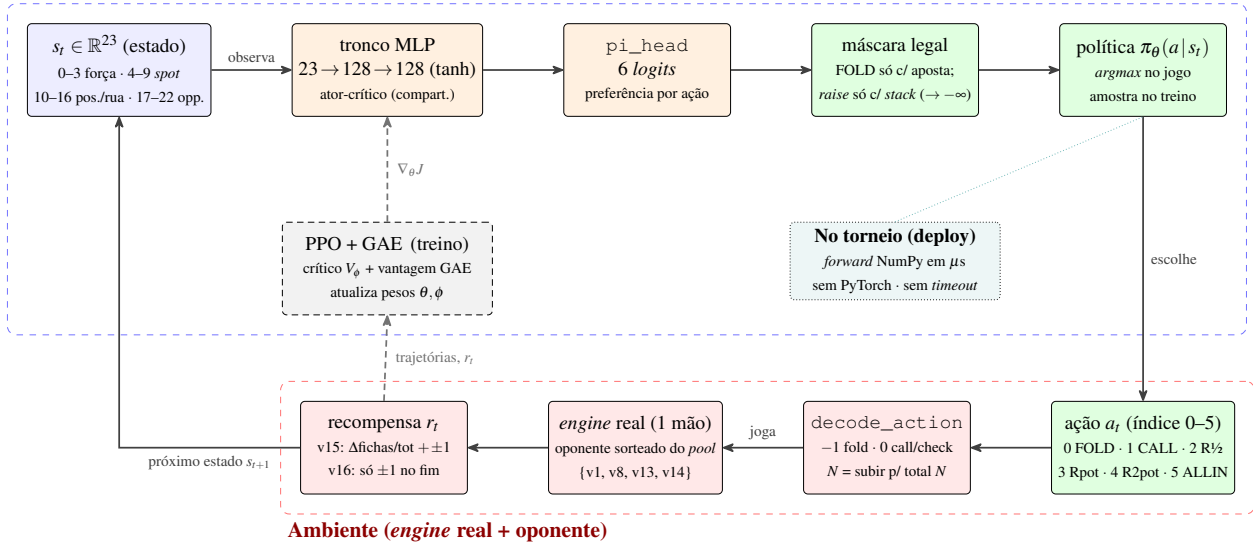


Figura 1. Hiper-fluxograma do Processo de Decisão de Markov. O ciclo $s_t \rightarrow a_t \rightarrow s_{t+1}$ liga o agente (acima) ao ambiente (abaixo); a espinha tracejada existe só no treino (PPO+GAE), e no torneio resta apenas o `forward NumPy`. A única diferença entre v15 e v16 está na caixa de recompensa r_t (densa vs. esparsa).

Tabela 4. Espaço de ações discreto.

Índice	Ação	Semântica
0	FOLD	desistir (só legal se há valor a pagar)
1	CALL	pagar / <i>check</i>
2	RAISE_HALF	subir $0,5 \times$ o <i>pot</i>
3	RAISE_POT	subir $1,0 \times$ o <i>pot</i>
4	RAISE_2POT	subir $2,0 \times$ o <i>pot</i>
5	ALLIN	apostar todo o <i>stack</i>

Uma *máscara de ações legais* zera a probabilidade de ações inválidas no *spot* (por exemplo, FOLD sem valor a pagar, ou *raise* sem *stack* suficiente), aplicando $-\infty$ aos respectivos *logits* antes da amostragem.

4.5. Função de Recompensa: a Única Diferença entre v15 e v16

Os dois agentes são treinados pelo mesmo procedimento, no mesmo *pool* de oponentes, com a mesma rede e os mesmos hiperparâmetros. A única variável manipulada é a recompensa:

v15 — recompensa densa (*chip*). A cada transição, o agente recebe a variação normalizada do próprio *stack*,

somada a um bônus terminal pelo resultado da partida:

$$r_t^{v15} = \underbrace{\frac{\Delta \text{fichas}_t}{\text{fichas totais}}}_{\text{denso}} + \underbrace{(+1 \text{ se venceu} \mid -1 \text{ se perdeu})}_{\text{terminal}}. \quad (5)$$

A soma telescópica de r_t ao longo da partida equivale ao ganho líquido de fichas mais o bônus, oferecendo um sinal de aprendizado *denso*, presente em quase toda decisão.

v16 — recompensa esparsa (*win_loss*). O termo denso é desligado; o agente só recebe sinal no fim da partida:

$$r_t^{v16} = \begin{cases} +1 & \text{se venceu a partida,} \\ -1 & \text{se perdeu a partida,} \\ 0 & \text{caso contrário.} \end{cases} \quad (6)$$

Essa recompensa alinha o objetivo *exatamente* ao que importa (vencer a partida), mas com um sinal muito mais fraco e tardio — um desafio clássico de atribuição de crédito.

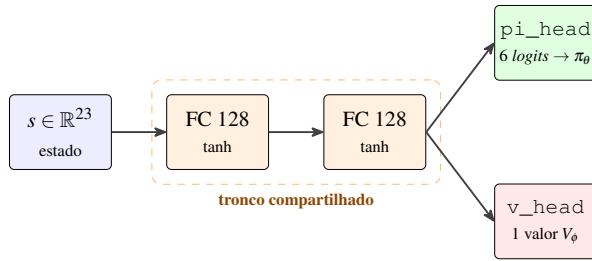


Figura 2. Rede Actor-Critic: tronco MLP compartilhado ($23 \rightarrow 128 \rightarrow 128$, tanh) com duas cabeças — a política π_θ (6 logits) e o valor V_ϕ (escalar). Idêntica em v15 e v16; só a recompensa de treino difere.

5. Implementação

5.1. Ambiente de Treino sobre a Engine Real

Para treinar sobre a *engine* verdadeira sem reescrevê-la, usamos *inversão de controle*: a partida real roda em uma *thread* dedicada, e o jogador do agente bloqueia em sua função de decisão, aguardando em uma fila a ação escolhida pelo treinador. Enquanto bloqueia, devolve ao treinador a tupla (atributos, máscara, fichas). Um ambiente vetorizado (`VecEnv`) executa N dessas partidas em paralelo lógico, com reinício automático, expondo ao PPO a interface familiar:

```

1 obs, mask = env.reset()
2 obs, mask, reward, done = env.step(
    action_idx) # arrays (N, ...)

```

Listing 1. Laço de interação vetorizado.

A cada partida, a posição (*dealer/não-dealer*) e o oponente são alternados/sorteados do *pool* {v14, v13, v8, v1}, expondo a política a estilos heterogêneos.

5.2. Arquitetura da Rede e Algoritmo

A política é uma rede *Actor-Critic* (*Multi-Layer Perceptron*) com tronco compartilhado de duas camadas ocultas de 128 unidades e ativação tanh, com cabeças separadas para a política (6 logits) e para o valor. O treino usa PPO com vantagem estimada por *Generalized Advantage Estimation* (GAE) (Seção 4.2). A Figura 2 esquematiza a rede e a Tabela 5 resume os hiperparâmetros.

Para execução no torneio, os pesos são exportados para um arquivo `.npz` e a inferência é feita por um *forward* 100% em *NumPy* (sem dependência de *PyTorch*), concluindo em microssegundos — muito abaixo do limite de tempo por decisão da *engine*.

5.3. Infraestrutura Robusta a Interrupções

O treino é projetado para sobreviver a interrupções: *checkpoints* periódicos com escrita atômica (arquivo temporário e `os.replace`), captura de SIGINT/SIGTERM (termina o *update* corrente, salva e sai) e retomada automática do último *checkpoint* — incluindo modelo, *optimizer*, contadores e estados dos geradores de números aleatórios. Cada experimento (v15 e v16) escreve em seu próprio diretório de *run*, com *log* e curva de evolução independentes.

Reprodutibilidade. Os treinos partiram de semente fixa ($seed = 0$) propagada para os geradores de *PyTorch*, *NumPy* e *Python*, e foram conduzidos em *Python* 3.14, *PyTorch* 2.12 (CUDA 12.6) e *NumPy* 2.3. Toda avaliação de força (Seção 6.1) usa 500 partidas por oponente em modo *greedy* (*argmax*), com posição (*dealer/não-dealer*) alternada a cada partida. Os *scripts* de treino, exportação e avaliação, com os comandos exatos de reprodução, acompanham o repositório do projeto (Seção 8).

6. Resultados

Tabela 6 resume os dois treinos, conduzidos do zero (pesos aleatórios) na mesma GPU (NVIDIA RTX 3050). A taxa de vitória (*WR*) aqui é a de *treino* — medida com *exploração* (amostragem de ações), sobre uma janela deslizante das últimas 500 partidas contra o *pool* de oponentes. Ela indica a curva de aprendizado, não a força real de jogo; esta é medida adiante (Seção 6.1), em partidas vencidas no modo *greedy*.

Custo computacional. Ambos os treinos rodaram numa única GPU NVIDIA RTX 3050 6 GB (*laptop*). O v15 (≈ 934 *updates*) levou $\approx 2,5$ h de GPU em sessão contínua; o v16, com cerca do dobro de *updates* (≈ 1830), exigiu $\approx 4\text{--}5$ h de computação efetiva. O custo da recompensa esparsa, portanto, foi concreto — aproximadamente o *dobro* de tempo de máquina para um ganho de ≈ 9 pontos de *win-rate* real (Tabela 7), um compromisso que se mostrou vantajoso.

As Figuras 3 and 4 mostram a evolução real de cada treino (geradas pelo *pipeline* de RL, painel `evolucao.png` de cada *run*). Cada figura traz seis painéis: a *win-rate* de treino (com exploração) e o melhor *checkpoint*, a entropia da política, a recompensa média por decisão e as perdas de política (PPO) e do crítico.

A leitura conjunta é direta: as duas curvas seguem a mesma assinatura de um treino PPO saudável (entropia decrescente, recompensa média subindo de negativa para

Tabela 5. Hiperparâmetros de treino (idênticos para v15 e v16).

Parâmetro	Valor	Parâmetro	Valor
Ambientes paralelos (N)	16	Desconto γ	0,999
Horizonte (T)	128	GAE λ	0,95
Épocas por <i>update</i>	4	<i>Clip</i> PPO ϵ	0,2
<i>Minibatches</i>	4	Taxa de aprendizado	3×10^{-4}
Coef. de entropia	0,01	Coef. de valor	0,5
<i>Max grad norm</i>	0,5	Camadas ocultas	128, 128

Tabela 6. Resumo dos treinos finais de v15 e v16.

Agente	Recompensa	<i>Updates</i>	Partidas	Melhor WR	WR final (janela)
v15 (<i>chip</i>)	densa	≈ 934	≈ 25 k	$\approx 75,2\%$	$\approx 71,4\%$
v16 (<i>win_loss</i>)	esparsa	≈ 1830	≈ 38 k	$\approx 82,8\%$	$\approx 78,2\%$

≈ 0 , perda do crítico caindo), mas o v16 *esparso* assenta em um patamar de *win-rate* mais alto — ao custo de cerca do dobro de *updates*.

6.1. Força real: partidas vencidas (500 por oponente, *greedy*)

A *win-rate* de treino é medida *com exploração* e contra todo o *pool*; a força real exige jogar *greedy* (*argmax*, como no torneio) em confronto direto. A Tabela 7 reporta o resultado medido em **partidas vencidas** num lote de **500 partidas por oponente**, alternando posição, usando os pesos de produção de cada bot (*weights_v15.npz* e *weights_v16.npz*). Como o *heads-up* sempre decide a partida, não há empates: vencidas + perdidas = 500 (a *win-rate* é apenas esse número dividido por 500).

Três fatos saltam da Tabela 7: (i) **ambos os agentes batem o v14** — até o v15 vence 342 de 500 partidas contra o melhor heurístico, e o RL *quebra a barreira dos 70%* que a linhagem manual ($v1 \rightarrow v14$) nunca cruzou; (ii) **o v16 é o mais forte e o mais regular**, vencendo mais partidas que o v15 contra *todos* os quatro oponentes — no total, **176 partidas a mais** em 2000 (79,7% contra 70,9%); e (iii) os maiores ganhos do v16 são contra os adversários mais duros, o **v14** (+57 partidas) e o **v13** (+53), exatamente onde a otimização do objetivo verdadeiro mais rende.

Significância estatística. Cada partida é um ensaio de Bernoulli, então a *win-rate* medida tem incerteza amostral quantificável por um intervalo de confiança binomial (aproximação normal), de meia-largura $1,96\sqrt{\hat{p}(1-\hat{p})/n}$. Para um oponente individual ($n = 500$, $\hat{p} \approx 0,8$), o IC 95% é de $\approx \pm 3,5$ pontos percentuais; para o total ($n = 2000$), encolhe para $\approx \pm 1,8$ ponto. Os

intervalos do v16 ($79,7\% \pm 1,8$) e do v15 ($70,9\% \pm 2,0$) são, portanto, *disjuntos*: a diferença de 8,8 pontos (176 partidas em 2000) excede em muito a soma das margens, de modo que a superioridade do v16 é **estatisticamente significativa**, e não fruto da variância amostral.

Confronto direto v15×v16. Os dois agentes de RL também jogaram *entre si*, no mesmo protocolo (*greedy*, 500 partidas, posição alternada). O resultado fecha a narrativa de forma inequívoca: o **v16 venceu 451 de 500 partidas** ($90,2\% \pm 2,6$) contra o v15. O domínio é *uniforme entre posições* — o v16 vence 89,6% como *dealer* e 90,8% como *não-dealer* —, indicando que a vantagem da recompensa esparsa não vem de um efeito posicional, mas de uma política globalmente mais sólida. O agente *disciplinado* não apenas vence mais partidas contra a linhagem antiga: domina diretamente o irmão *ganancioso*.

7. Discussão

O resultado mais notável, visível ao comparar as Figuras 3 and 4, é que a recompensa *esparsa* (v16) assenta em um patamar de *win-rate* mais alto que a densa (v15), apesar do sinal de aprendizado muito mais fraco. Duas explicações se destacam:

1. **Alinhamento de objetivo.** A recompensa densa (Eq. (5)) premia acumular fichas a cada decisão, o que nem sempre coincide com vencer a partida — cujo desfecho se concentra nas últimas mãos (regime *push/fold*). A recompensa esparsa (Eq. (6)) otimiza *diretamente* a métrica de interesse, sem o viés de um *proxy* intermediário.
2. **Orçamento de treino.** O v16 recebeu cerca do dobro de *updates* do v15 (Tabela 6), compensando

Evolução do treino — v15_chip (reward=chip) (PPO) | update 934, WR 71.4%, melhor 75.2%, entropia 1.16

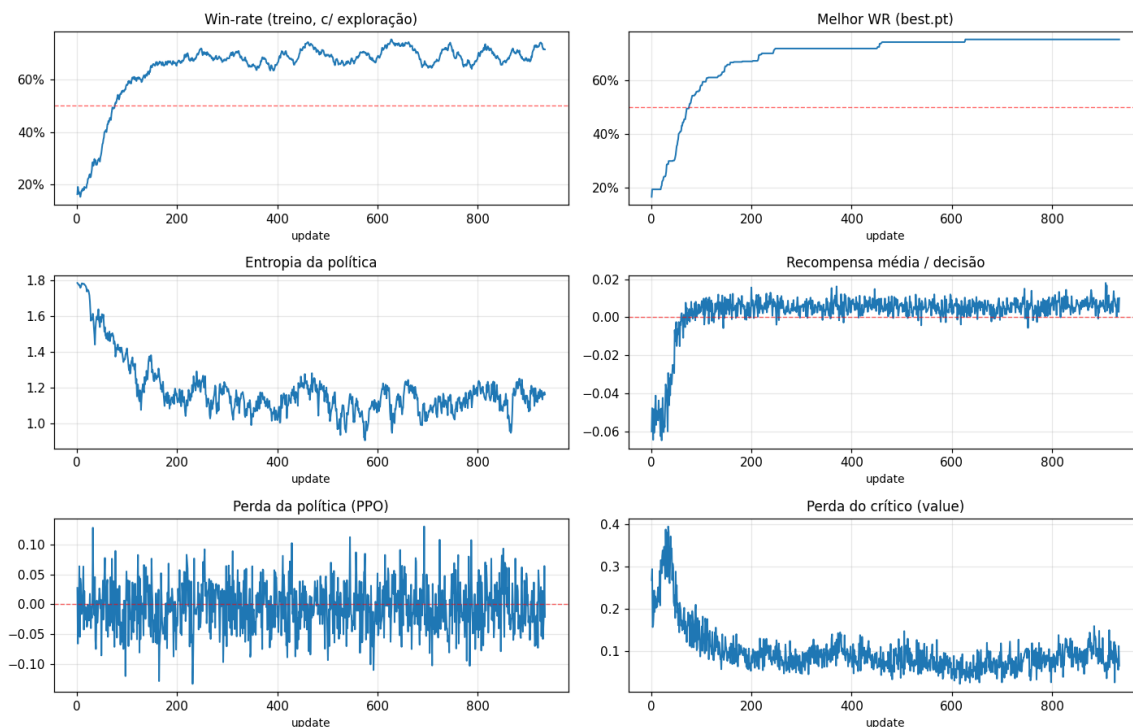


Figura 3. Evolução do treino do v15 (recompensa densa, *chip*). A *win-rate* cruza 50% por volta do *update* 70 e estabiliza na faixa de 70–75% contra o *pool*; a entropia cai de $\approx 1,8$ para $\approx 1,1$ (a política deixa de ser quase uniforme) e as perdas convergem.

o sinal esparsos. Isso é coerente com a literatura de recompensas esparsas [8]: elas exigem mais amostras, mas, quando bem exploradas, evitam soluções “gananciosas” de curto prazo.

O objetivo do projeto foi atingido: vencer o v14. Na avaliação *greedy* (Tabela 7), o objetivo declarado deste trabalho — *vencer o v14* — foi alcançado pelos dois agentes de RL: o v15 vence 342/500 e o v16 399/500 partidas contra o melhor heurístico da linhagem. O RL quebrou a barreira de 70% que o ajuste manual (v1→v14) nunca cruzou. E a recompensa esparsa do v16 rendeu mais justamente onde importa: os maiores saltos sobre o v15 são contra o v14 (+57 partidas) e o v13 (+53). Em outras palavras, a recompensa “fácil” (densa) otimiza um substituto (fichas) que *não* é exatamente o que se quer (vencer a partida) — e isso aparece no número final.

O “melhor” *checkpoint* denso joga pior: evidência direta do *proxy*. Um achado revela o problema da recompensa densa de forma contundente. O treino salva

dois *checkpoints*: o último (*latest.pt*) e o de maior *win-rate* de treino (*best.pt*). A Tabela 8 avalia os dois *checkpoints* de cada agente na mesa (*greedy*, 500 partidas por oponente). O contraste é gritante: para o v15 *denso*, o *best.pt* — “melhor” por acumular mais fichas contra o *pool* — joga *pior*, **desabando para 134/500 partidas contra o v14** (26,8%) e perdendo para quase toda a linhagem (861/2000 no total, contra 1379/2000 do *latest.pt*). Maximizar o *proxy* (fichas) não só deixa de ajudar: *piora* o objetivo real (vencer). Por isso o v15 implantado usa o *latest.pt*. Para o v16 *esparso* a escolha se inverte — o *best.pt* é o mais forte (1606/2000) e é o implantado, ao passo que seu *latest.pt* regrediu (871/2000), reflexo da maior variância de fim de treino sob sinal esparsos. Em ambos os casos, o *checkpoint* de maior *win-rate* de treino só coincide com a força real quando a recompensa está alinhada ao objetivo: é a confirmação mais nítida de que recompensa densa e objetivo verdadeiro podem divergir. Esse é exatamente o regime que a teoria de *reward shaping* adverte: apenas transformações de recompensa baseadas em potencial preservam a política ótima; re-

Evolução do treino — v16_sparse (reward=win_loss) (PPO) | update 1830, WR 78.2%, melhor 82.8%, entropia 1.08

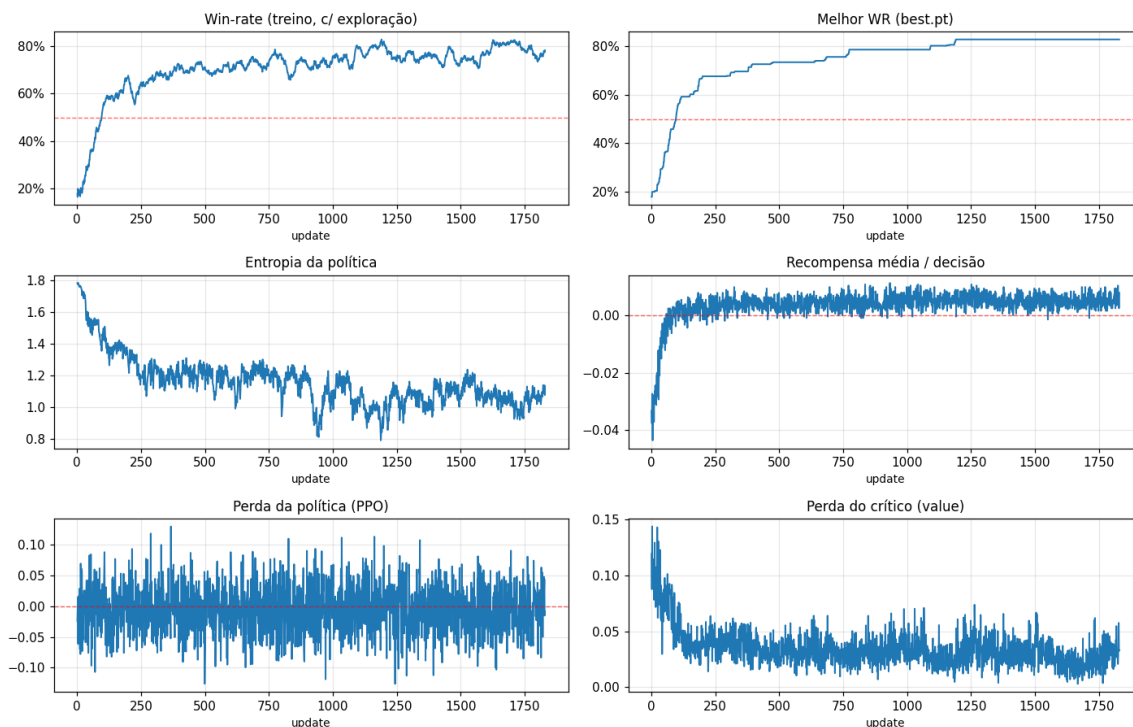


Figura 4. Evolução do treino do v16 (recompensa esparsa, *win_loss*). Apesar do sinal mais fraco (só ± 1 no fim da partida), a *win-rate* sobe acima da do v15 e estabiliza em torno de 78%, com melhor *checkpoint* em $\approx 82,8\%$; o treino exigiu cerca do dobro de *updates* do v15.

compensas densas arbitrárias, como o fluxo de fichas, podem enviesar o aprendizado [7].

O que os pesos finais aprenderam. Pesos de um MLP não se leem diretamente, então os *projetamos em comportamento*. Montamos estados reais (GameView) varrendo a força da mão contra o *stack* efetivo, rua a rua, e registramos a ação *modal* que cada agente escolhe na *abertura* — quando é o primeiro a agir, sem aposta a pagar. O resultado é a Figura 5: a cor de cada célula é a ação predominante naquele *spot*, de passar/*check* (azul) a *all-in* (vermelho). Como o mesmo estado é dado aos quatro agentes, o mapa isola exatamente *como cada um decide* — e dispensa interpretar um único número de peso.

A leitura por linha revela três *temperamentos* bem distintos. Os **heurísticos** (v1, v8) são *tight*: dominam o azul (passar) e reservam o amarelo (meio-pote) às mãos fortes, sem *já* acionar o vermelho (*all-in*) na abertura — em *nenhuma* das três ruas. É a cautela de um “livro” de jogador, que arrisca o *stack* só depois de ver mais cartas. O **v15 denso** é o extremo oposto: o azul

quase desaparece; ele abre com *raise* em quase todo o *range* e parte para o *all-in* cedo, com pouca força. É a assinatura de quem foi premiado por *acumular fichas* a cada mão — agressão indiscriminada, que maximiza o *proxy* mas se expõe demais. Já o **v16 esparsa** reencontra o equilíbrio: preserva uma ampla faixa passiva (azul) e só compromete o *stack* (vermelho) com as mãos realmente fortes, num limiar quase *constante* entre as ruas — uma regra de *commit* simples e estável, a assinatura de quem foi premiado apenas por *vencer a partida*.

O contraste fecha o argumento central do trabalho. Mesma rede, mesmo *pool*, mesmos hiperparâmetros: só a recompensa muda, e ela sozinha esculpe estilos opostos — do agressivo (v15) ao disciplinado (v16). E é o lado disciplinado que converte esse estilo em vitórias, nas +176 partidas da Tabela 7 e no 451/500 do confronto direto. Em suma, no nosso experimento controlado é a *recompensa*, e não a arquitetura, que define o comportamento aprendido.

Ameaças à validade. A *win-rate* das figuras é uma janela móvel *durante* o treino, contra os mesmos opo-

Tabela 7. Partidas vencidas (de 500 por oponente, *greedy*). O melhor de cada linha em negrito; o v16 usa o melhor *checkpoint* (*best .pt*).

Oponente	v15 (denso)	v16 (esparso)	Δ (partidas)
v14 (<i>top</i> heurístico)	342/500	399/500	+57
v13	343/500	396/500	+53
v8	377/500	394/500	+17
v1 (<i>benchmark</i>)	355/500	404/500	+49
Total	1417/2000 (70,9%)	1593/2000 (79,7%)	+176

Tabela 8. Força real de cada *checkpoint* (*greedy*, 500 partidas/opponente, contra o *pool* {v14, v13, v8, v1}). O *checkpoint* implantado de cada agente em negrito. Medição independente: pequenas diferenças vs. a Tabela 7 refletem variância amostral.

Agente	<i>Checkpoint</i>	vs v14	Total (de 2000)
v15 (denso)	latest (implantado)	312/500	1379 (69,0%)
	best (pico de WR de treino)	134/500	861 (43,1%)
v16 (esparso)	best (implantado)	401/500	1606 (80,3%)
	latest	142/500	871 (43,6%)

nentes do *pool*; ela pode superestimar o desempenho contra adversários não vistos. Triagens com poucas partidas têm alta variância ($\pm$ vários pontos percentuais), de modo que comparações finais demandam milhares de partidas. Por fim, o *pool* de treino é heurístico e estático; um oponente que se adapta ativamente à política aprendida não foi avaliado.

8. Conclusão e Trabalhos Futuros

A linhagem heurística atingiu seu auge na **Fase 2** do campeonato, com o **v14 campeão** e o **v13 vice-campeão** (Tabela 2) — um teto alto a ser superado. Sobre ele, apresentamos os agentes v15 e v16, treinados por PPO na *engine* real, mantendo tudo constante exceto a recompensa para isolar seu efeito. Medidos em 500 partidas por oponente (Tabela 7), *ambos* superam o v14, e a recompensa esparsa (v16) é a mais forte — 1593/2000 (79,7%) contra o conjunto e 399/500 contra o próprio v14, além de bater o v15 no confronto direto (451/500). O RL destravou a barreira de 70% que o ajuste manual nunca cruzou e reforçou que alinhar a recompensa à métrica final compensa, mesmo ao custo de mais amostras.

Direções futuras incluem: (i) *self-play* com *pool* de oponentes dinâmico, à la NFSP [9], para mitigar o *overfitting* ao conjunto estático; (ii) avaliação cruzada extensa (milhares de partidas) contra adversários não vistos; (iii) *reward shaping* baseado em potencial [7] como meio-termo testável entre as recompensas densa e esparsa, preservando a otimalidade da política; e (iv) investigar arquiteturas recorrentes para capturar melhor

o histórico da mão.

Disponibilidade de Código

Todo o código deste trabalho — os agentes (`players/`), o *pipeline* de RL (`rl/`: ambiente, treino PPO, exportação de pesos e avaliação) e os relatórios — está disponível publicamente no repositório do projeto [12].

Referências

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, e O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [2] J. Schulman, P. Moritz, S. Levine, M. Jordan, e P. Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *ICLR*, 2016.
- [3] R. S. Sutton e A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2ª ed., 2018.
- [4] N. Brown e T. Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456):885–890, 2019.
- [5] N. Brown e T. Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.
- [6] M. Zinkevich, M. Johanson, M. Bowling, e C. Piccione. Regret Minimization in Games with Incomplete Information. *Advances in Neural Information Processing Systems (NeurIPS)*, 2007.
- [7] A. Y. Ng, D. Harada, e S. Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. *Proc. ICML*, p. 278–287, 1999.

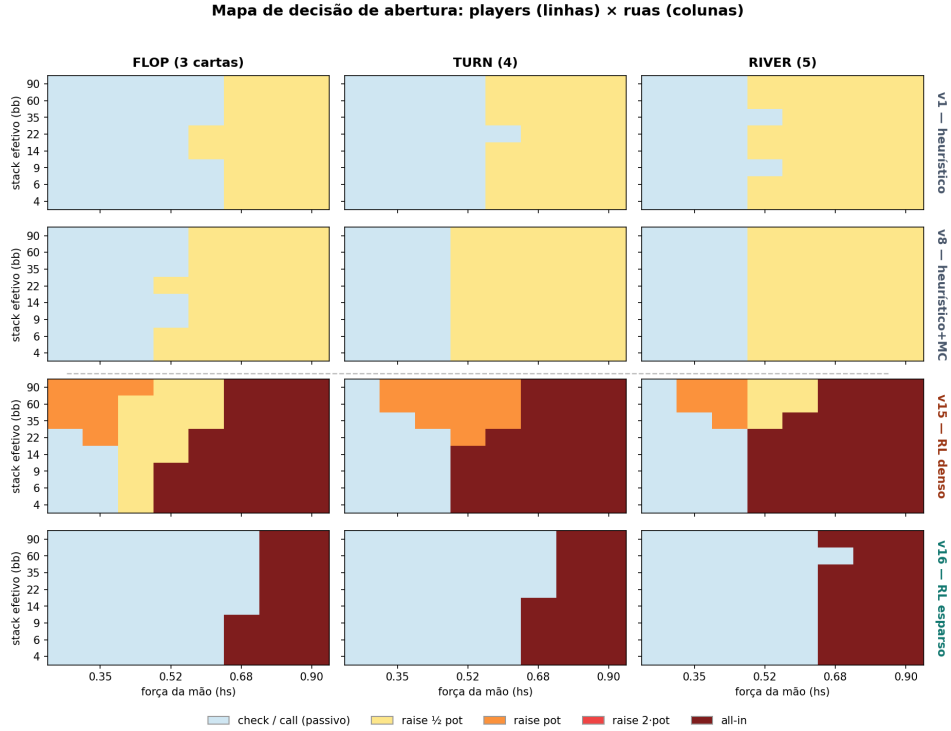


Figura 5. Mapa de decisão na abertura: ação *modal* por célula em função da força da mão (hs , eixo x) e do *stack* efetivo (bb , eixo y); cores de azul (passar) a vermelho (*all-in*), conforme a legenda. Linhas: agentes heurísticos (v1, v8) acima da linha tracejada e aprendidos por RL (v15, v16) abaixo; colunas: as ruas (*flop/turn/river*). Interpretação no texto.

- [8] M. Andrychowicz, F. Wolski, A. Ray, et al. Hindsight Experience Replay. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] J. Heinrich e D. Silver. Deep Reinforcement Learning from Self-Play in Imperfect-Information Games. *arXiv:1603.01121*, 2016.
- [10] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [11] D. Silver, T. Hubert, J. Schrittwieser, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [12] HackathonYamadaITAJr — código dos agentes de poker e do *pipeline* de RL (v1–v16). <https://github.com/IvanYamasaki/HackathonYamadaITAJr>, 2026.